

FPGA-based SoC Design

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

More on Introduction to SystemVerilog

Today's Agenda

Intended topics for today's session

- More on lexical rules, data types, user defined data types, logic values, number representation, packages

FPGA-based SoC Design

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

More on Introduction to SystemVerilog

Objectives

By the end of this lecture you will be able to ...

- ... apply a wider range of SystemVerilog features in order to describe and implement more complex digital circuitries.

FPGA-based SoC Design

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

More on Introduction to SystemVerilog

Recommended Readings

Textbooks, Application Notes, White Papers ...

- Sutherland, S., “RTL Modeling with SystemVerilog for Simulation and Synthesis: Using SystemVerilog for ASIC and FPGA Design”, CreateSpace Independent Publishing Platform, 2017
- Spear, C., “SystemVerilog for Verification: A Guide to Learning the Testbench Language Features”, Springer, 3rd edition, 2012.

If I designed a computer with 200 chips, I tried to design it with 150. And then I would try to design it with 100. I just tried to find every trick I could in life to design things real tiny.

Stephen Gary Wozniak, 'the Woz', co-founder of Apple Inc. and *computer pioneer*

Introduction to SystemVerilog HDL – Part #6

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

SystemVerilog - Lexical Rules

FPGA-based SoC Design

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Lexical Rules

```
1.  /*  
2.      Multiline comment ...  
3.  */  
4.  module xor_8_reduction(  
5.      input logic [3:0] d, output logic q  
6.  );  
7.      // Single line comment  
8.      assign q = ^d;  
9.  
10. endmodule
```

1/1

[SystemVerilog] Source Code: xor_8_reduction.sv

Note

- SystemVerilog is **case-sensitive**, therefore upper and lower case letters have different meanings: D is not d!
- Moreover, SystemVerilog is a free formatting language, what means that spaces can be added freely to format the source code.
- Comments in SystemVerilog are just like those in C.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

SystemVerilog - Data Types

IEEE Standard 1800-2012

Basic Data Types

bit	User defined vector size; defaults to unsigned
byte	8-bit signed integer or ASCII character
shortint	16-bit signed integer
int	32-bit signed integer
longint	64-bit signed integer

Table: **2-state** Data Types (only 0 or 1)

logic	User defined vector size; defaults to unsigned
reg	User defined vector size; defaults to unsigned
integer	32-bit signed integer
time	64-bit unsigned integer

Table: **4-state** Data Types (0,1,x,z)

Note

- 2-state data types simulate faster and consume less memory than 4-state data types
 - However, 2-state data types in simulation may not accurately model the actual hardware implementation
-
- **This new data type is intended to remove confusion associated with the reg data type in Verilog.** It was sometimes mistakenly understood that a register of the given variable name should be created in hardware.
 - However, there is no correlation between using a reg variable and the hardware inferred in Verilog.
 - **The context of the reg variable** determines whether combinational or sequential logic is inferred.

Basic Data Types

bit	User defined vector size; defaults to unsigned
byte	8-bit signed intefer or ASCII character
shortint	16-bit signed integer
int	32-bit signed integer
longint	64-bit signed integer

Table: 2-state Data Types (only 0 or 1)

logic	User defined vector size; defaults to unsigned
reg	User defined vector size; defaults to unsigned
integer	32-bit signed integer
time	64-bit unsigned integer

Table: 4-state Data Types (0,1,x,z)

Assigning the same value to all bits of a variable ...

```
1. unsigned int data;  
2. ...  
3. assign data = `0;
```

1/1

[SystemVerilog] Source Code Excerpt

Note

- The respective value is assigned to all bit positions

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Strings and String Methods in SystemVerilog

IEEE Standard 1800-2012

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Strings

```
1. module demo_testbench();  
2.    // string declaration and initialization  
3.    string message = "Simulation starts!";  
4.  
5.    initial  
6.        begin : testbench_scheduler  
7.            $display("%s",message);  
8.            ...  
9.        end  
10.  
11. endmodule
```

1/1

[SystemVerilog] Source Code Excerpt

Note

- The data type string represents an ordered set of characters.
- The length of a string variable is equal to the number of characters in the set.
- The length can dynamically vary throughout the course of a simulation.
- Commonly used in testbenches

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Strings Methods available

```
1. module demo_design #(parameter string mode = "standard") (
2.     input logic clk, reset_n,
3.     input logic [7:0] data
4.     ...
5. );
6.
7. localparam SETTINGS = (mode.icompare("standard") == 2) ? 12 : 15;
8.
9. ...
```

1/1

built-in method notation

[SystemVerilog] Source Code Excerpt

Note

- The Intel Quartus Prime software supports string data types for parameters.
- SystemVerilog also supports a number of special methods to work with strings. .
- These methods use the built-in method notation.

Basic string methods

- `str.len()`
- `str.putc()`
- `str.getc()`
- `str.tolower()`
- `str.compare(s)`
- `str.icompare(s)`
- `str.substr (i, j)`

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Increment and Decrement Assignment Operators

IEEE Standard 1800-2012

Increment and Decrement Assignment Operators

```
1. // Post-Increment: i is assigned to data and then incremented
2. data = i++;
3.
4. // Pre-Increment: i is first incremented and then its new value is assigned to data
5. data = ++i;
6.
7. // Post-Decrement: i is assigned to data and then decremented
8. data = i--;
9.
10. // Pre-Decrement: i is first decremented and then its new value is assigned to data
11. data = --i;
```

1/1

[SystemVerilog] Source Code Excerpt

Note

- SystemVerilog includes the C increment and decrement assignment operators.
- These increment and decrement assignment operators behave as **blocking assignments**.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Compound Assignment Operators

IEEE Standard 1800-2012

Compound Assignment Operators

```
1. // %= operator
2. data %= range;           // this is equivalent to: data = data % range
3.
4. // <=> operator
5. data <=> scale;           // this is equivalent to: data = data << scale
6.
7. // >>= operator
8. data >>= scale;           // this is equivalent to: data = data >> scale
9.
10. // &= operator
11. data &= mask;            // this is equivalent to data = data & mask
12.
13. // ^= operator
14. data ^= mask;            // this is equivalent to data = data ^ mask
15.
16. // |= operator
17. data |= mask;            // this is equivalent to data = data | mask
```

1/1

[SystemVerilog] Source Code Excerpt

Compound Assignment Operators

```
1. // += operator
2. data += adc_sample;    // this is equivalent to: data = data + adc_sample
3.
4. // -= operator
5. data -= adc_sample;    // this is equivalent to: data = data - adc_sample
6.
7. // *= operator
8. data *= scale;         // this is equivalent to: data = data * scale
9.
10. // /= operator
11. data /= scale;         // this is equivalent to data = data / scale
```

1/1

[SystemVerilog] Source Code Excerpt

Note

- SystemVerilog includes the C compound assignment operators.

Compound Assignment Operators

```
1. always_comb begin
2.     q_alu = op_a;
3.     unique case (op_code)
4.         ADD:    q_alu += op_a;
5.         SUB:    q_alu -= op_a;
6.         MUL:    q_alu *= op_a;
7.         DIV:    q_alu /= op_a;
8.         MOD:    q_alu %= op_a;
9.         B_AND:  q_alu &= op_a;
10.        B_OR:   q_alu |= op_a;
11.        B_XOR:  q_alu ^= op_a;
12.        SL:     q_alu <<= op_a;
13.        SR:     q_alu >>= op_a;
14.        ASL:    q_alu <<<= op_a;
15.        ASR:    q_alu >>>= op_a;
16.    endcase
17. end
```

1/1

Note

- A dedicated use case – ALU datapath
- All assignment operator is semantically equivalent to a blocking assignment.
- Something in between:

Arithmetic right shift (>>>) - shift right specified number of bits, fill with value of sign bit if expression is signed, otherwise fill with zero,

Arithmetic left shift (<<<) - shift left specified number of bits, fill with zero.

Whereas **logical shift (<<, >>)** always fill the vacated bit positions with zeroes.

[SystemVerilog] Source Code Excerpt

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Numeric Constants

IEEE Standard 1800-2012

Numeric Constants

Numbers	Bits	Base	Value	Stored
3'b101	3	2	5	101
'b11 **	?	2	3	00 ... 011
8'b11	8	2	3	0000011
8'b1010_1011	8	2	171	10101011
6'o42	6	8	34	100010
3'd6	3	10	6	110
8'hAB	8	16	171	10101011
42	?	10	42	00 ... 0101010

** Automatic scaling. Often used in parametrized modules.

Table: Numeric constants in SystemVerilog

Note

General form: <size> <signed> <radix> value ["<>" indicates optional part]

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

User-Defined Data Types

IEEE Standard 1800-2012

User-Defined Data Types

```
1. // type definitions
2. typedef unsigned int uint32;
3. typedef logic [15:0] data_bus;
4. ...
```

1/1

[SystemVerilog] Source Code Excerpt

Note

- Typedef allows users to create their own names for type definitions that they will use frequently in their code.
- It can be applied for a single or multiple modules:
 - **Local declaration:** If typedef is used inside of a module, then it is only visible within that module
 - **External declaration:** If typedef is used outside of any module, then it can be visible within several modules.

Structures

```
1. struct {  
2.     logic [7:0] data_word;  
3.     logic even_parity_bit;  
4. } data_struct;  
5.  
6. data_struct data_struct_new;  
7.  
8. assign data_struct_new.data_word      = data[7:0];  
9. assign data_struct_new.even_parity_bit = ^data[7:0];
```

1/1

[SystemVerilog] Source Code Excerpt

Note

- A structure is a user-defined collection of various datatypes. Each of the constituent members of a structure is also called its field. These fields can be either standard datatypes or, they can be user-defined types (using typedef) and, possibly, another structure.
- Structures are used to group inter-related data.
- We use the 'dot notation' to access a particular element of a structure.

Structures and User-Defined Data Types

```
1. typedef struct {  
2.     logic [7:0] data_word;  
3.     logic even_parity_bit;  
4. } data_struct;  
5.  
6. data_struct my_data = {8'b00000000, 1'b0};  
7.  
8. assign my_data.data_word    = data[7:0];  
9. assign my_.even_parity_bit = ^data[7:0];
```

1/1

[SystemVerilog] Source Code Excerpt

Note

- We can also combine struct and typedef to create new data types

Initializing Structures

```
1. typedef struct {  
2.     logic a, b, c;  
3.     int    d, e, f;  
4. } my_struct;  
5.  
6. my_struct new = {1'b1, 1'b1, 1'b0, 1, 2, 3};
```

1/1

[SystemVerilog] Source Code Excerpt

Note

- Initial values can be specified within curly braces by listing them in the order that they are declared within the structure

Structures Passed Through Module Ports

```
1. typedef struct {  
2.     logic [7:0] data_word;  
3.     logic even_parity_bit;  
4. } data_struct;  
5.  
6.  
7. module data_proc (    input logic clk, reset_n,  
8.                     input data_struct,  
9.                     ...
```

1/1

[SystemVerilog] Source Code Excerpt

Note

- In order to be able to pass structures through the ports of a module, we must place the typedef struct statement outside of the module.
- Then, you can pass variables of that type into and out of the module

Enumerations

```
1. // enumerated data types
2. enum {FETCH,DECODE,EXECUTE,WRITEBACK} proc_state;
3. enum bit {TRUE, FALSE} return_state;
4. ...
```

1/1

[SystemVerilog] Source Code Excerpt

0

...

```
5. enum {FETCH,DECODE,EXECUTE,MEM,WRITEBACK} cpu_state;
```

1

```
6. enum logic [1:0] {FETCH,DECODE, EXECUTE};
```

Note

- SystemVerilog also introduces enumerated types.
 - Enumerations allow you to define a data type whose values have names and thus improves the readability of the code.
 - Such data types are appropriate and useful for representing state values, opcodes and other such non-numeric or symbolic data.
-
- Note that if the type is not defined in 'enum' then the type will be set as 'integer' (32-bit int, 2-state values) by default, which may use extra memory unnecessarily.
 - The first name listed gets the value 0.
 - The second name listed gets the value 1, ...

Enumerations used with User-Defined Data Types

```
1. typedef enum {add, sub, mult, div } instruction; 1/1
2. instruction c;
3. ...
```

[SystemVerilog] Source Code Excerpt

Note

- Enumerations are often used in conjunction with typedef.

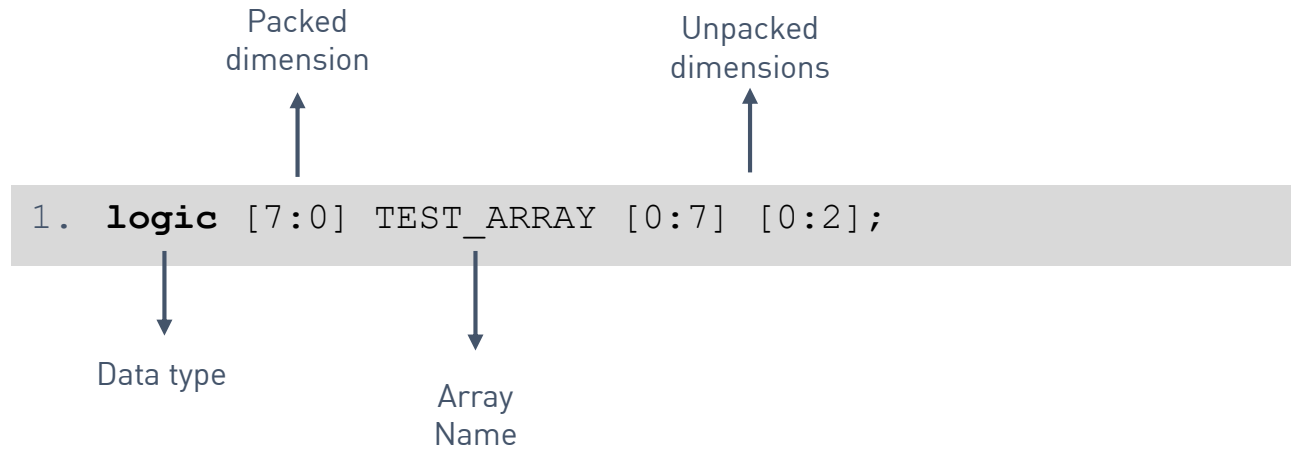
Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Arrays in SystemVerilog

IEEE Standard 1800-2012

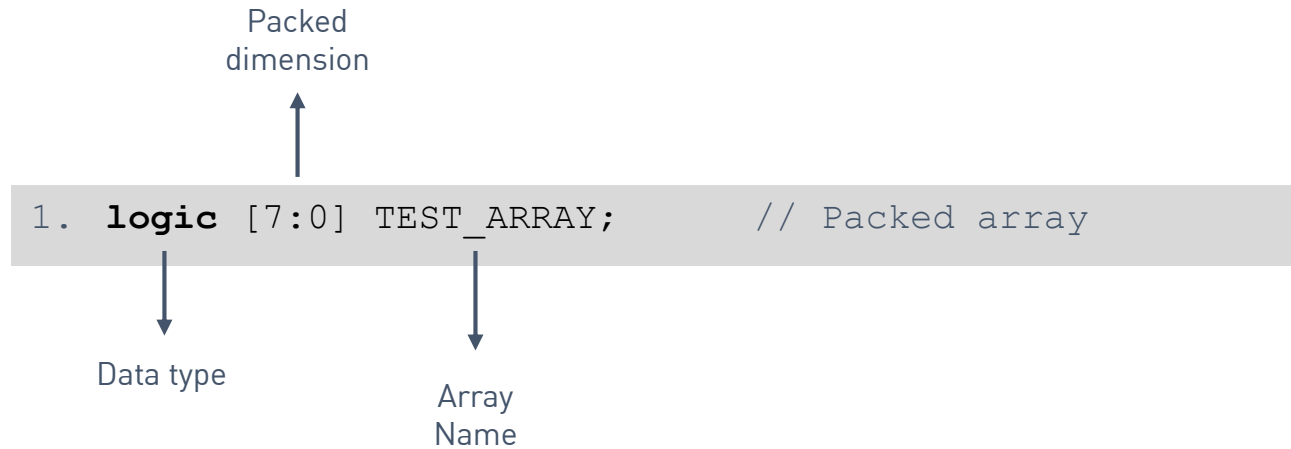
Arrays



Please note

- The term packed array is used to refer to the dimensions declared before the data identifier name.
- The term unpacked array is used to refer to the dimensions declared after the data identifier name.
- A packed array is guaranteed to be represented as a contiguous set of bits.
- An unpacked array may or may not be so represented as a contiguous set of bits.

Arrays



Note

- **One dimensional packed array is also denoted as a vector**
- Packed arrays are guaranteed to be laid out contiguously in memory.
- Packed arrays are restricted to the "bit" types (bit, logic, int, ...)
- Write access:

```
2. assign TEST_ARRAY = 12;
```

Arrays

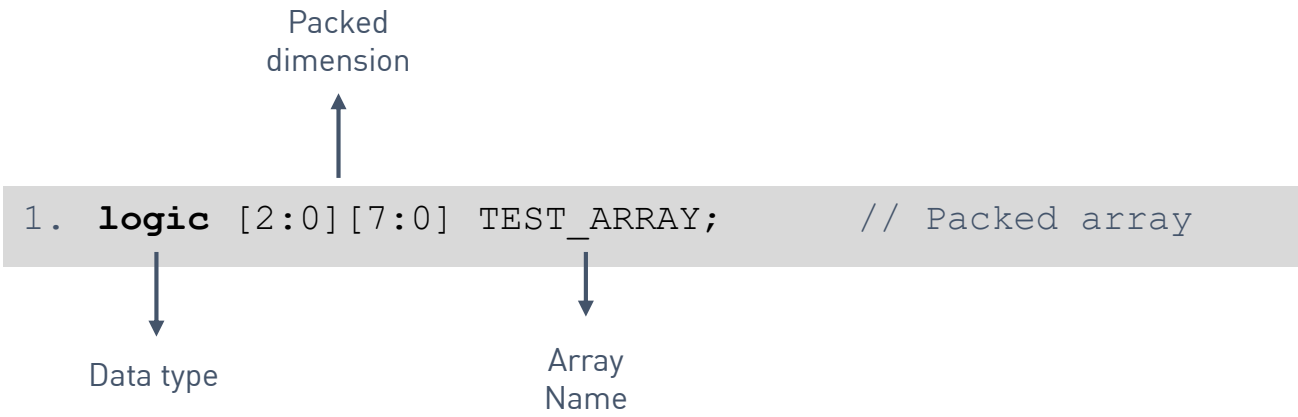
```
1. logic [7:0] A; assign A[7:5] = 3'b101;
```



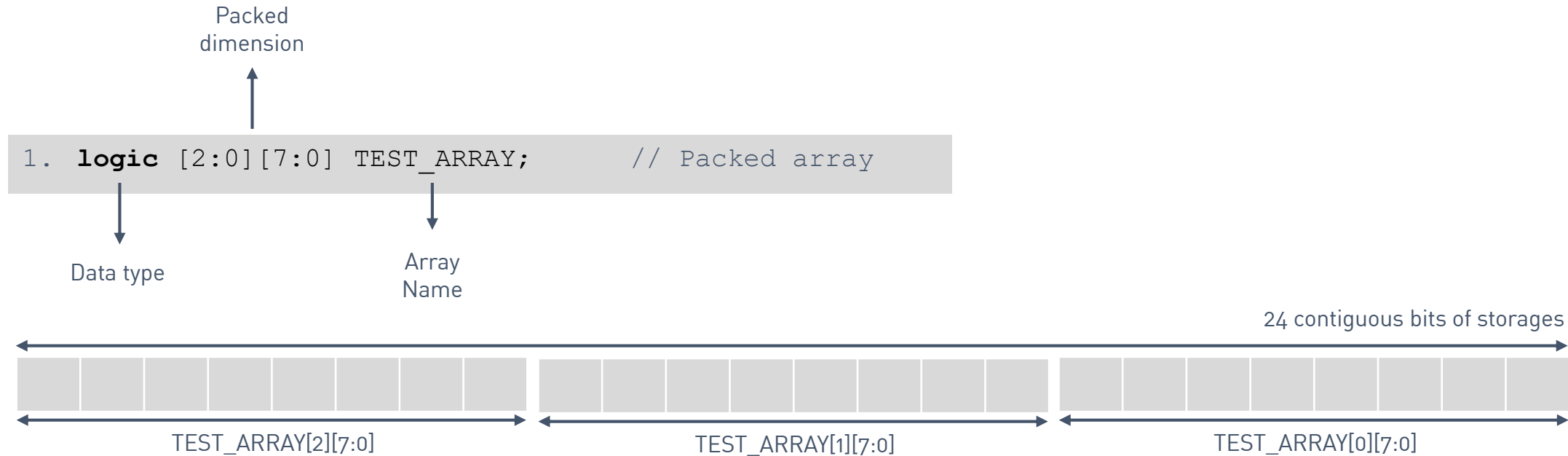
Note

- Access parts of a vector ...

Arrays



Arrays



Note

- Two-dimensional packed array.
- **This represents still a set of contiguous bits in memory, however segmented into smaller groups.**
- A packed array is a mechanism for subdividing a vector (24-bit in the upper example) into sub-fields (8-bit in the upper example), which can be conveniently accessed as array elements.

Arrays

```
1. logic [2:0][7:0] TEST_ARRAY = 24'd0;
```

Note

- When declaring a packed array, you can optionally include an initial value as part of the declaration.
- Alternatively, it is possible to set the initial values as follows:

```
2. logic [2:0][7:0] TEST_ARRAY = {8'd1, 8'd2, 8'd3};
```

- Using the replication operator:

```
3. logic [2:0][7:0] TEST_ARRAY = {3{8'hAF}};
```

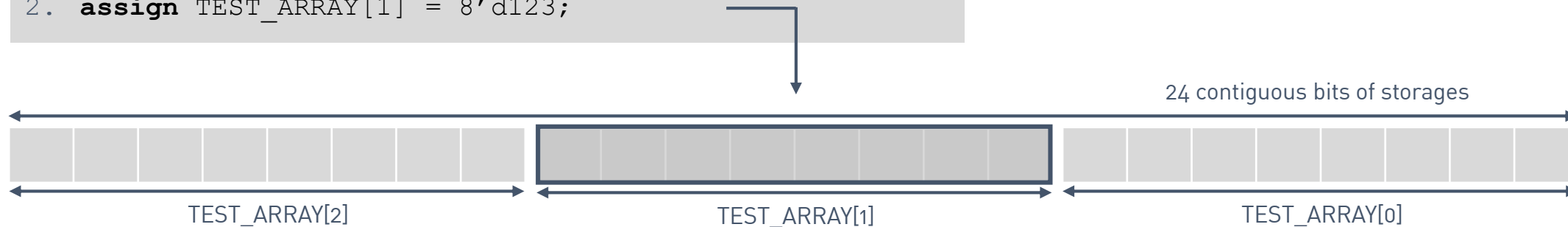
Arrays

```
1. logic [2:0][7:0] TEST_ARRAY;      // Packed array
```

Note

- Multi-dimensional packed array.
- This represents still a set of contiguous bits in memory, however it is also segmented into smaller groups.

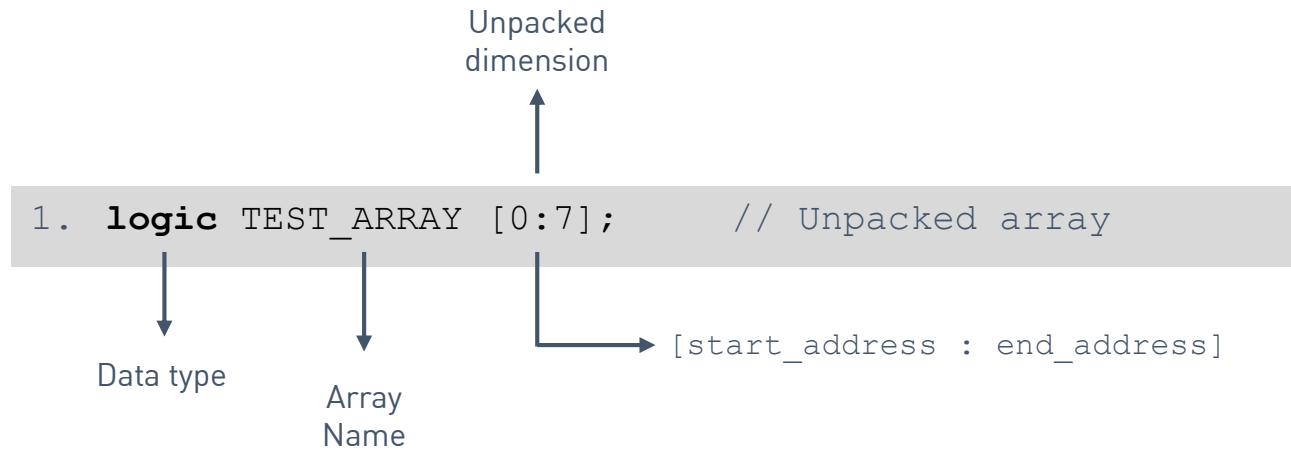
```
2. assign TEST_ARRAY[1] = 8'd123;
```



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Arrays



Note

- Single dimension unpacked array.
- Alternative way of expressing the unpacked dimension.

```
2. logic TEST_ARRAY [8]; // Unpacked array
```



Arrays

```
1. assign TEST_ARRAY = 8'd123;      // Unpacked array
```

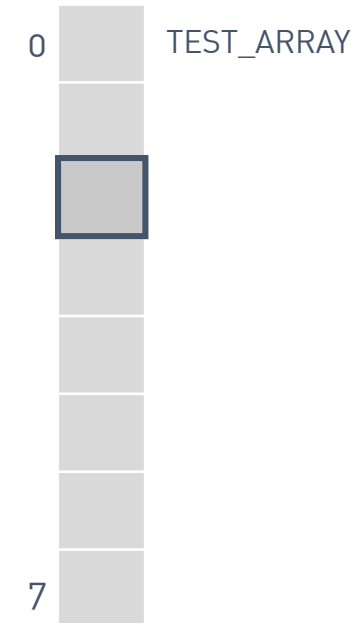
Note

- The previous expression leads to a SystemVerilog error:

Error [10928]: SystemVerilog error at test_design.sv(19): packed array type cannot be assigned to unpacked array type - types do not match

- The following code examples accesses the fourth location of the TEST_ARRAY:

```
2. assign TEST_ARRAY[2] = 1'b1;
```



Introduction to SystemVerilog

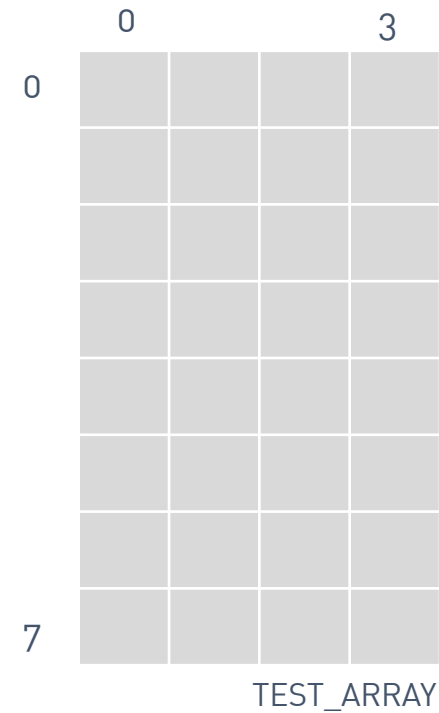
h_da – fbeit - FPGA-based SoC Design

Arrays



Note

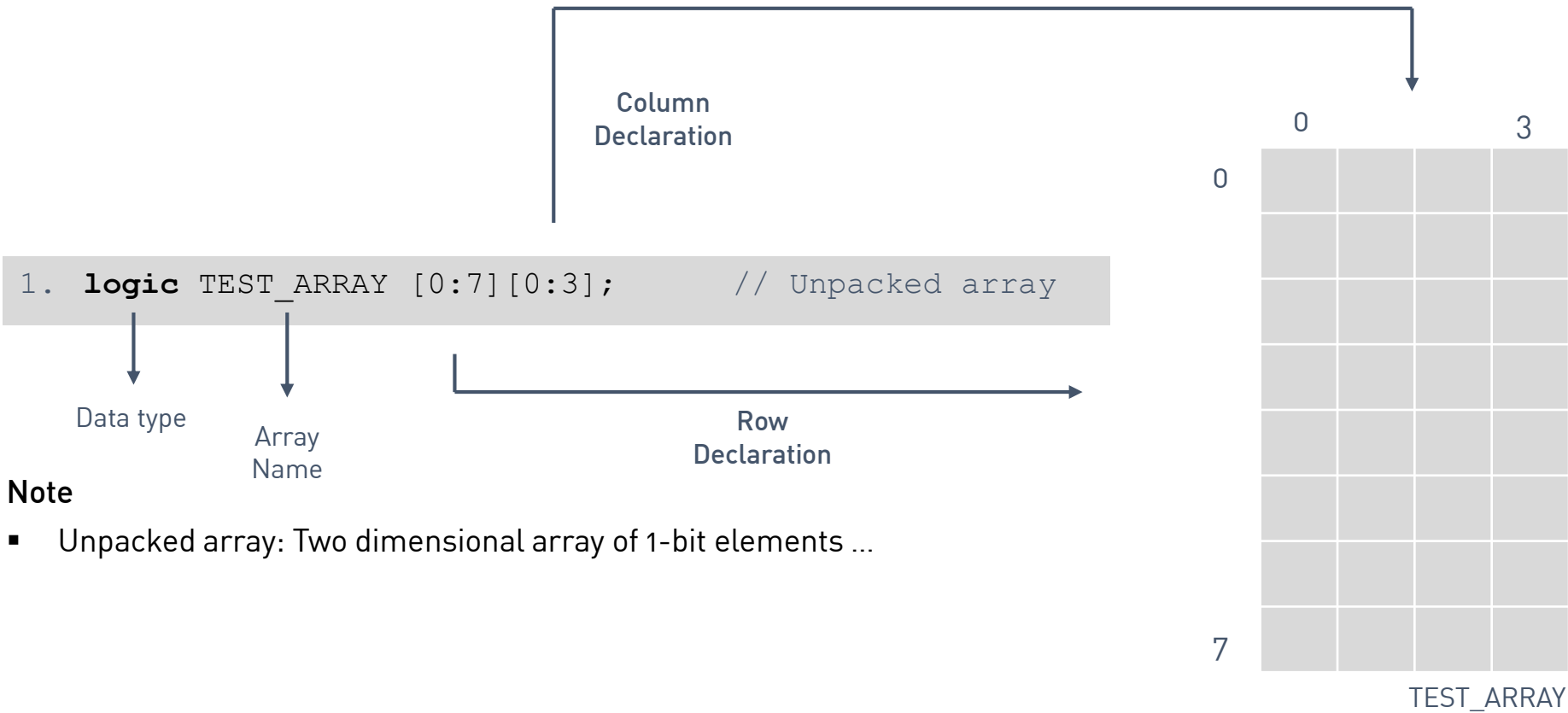
- Unpacked array: Two dimensional array of 1-bit elements ...



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Arrays



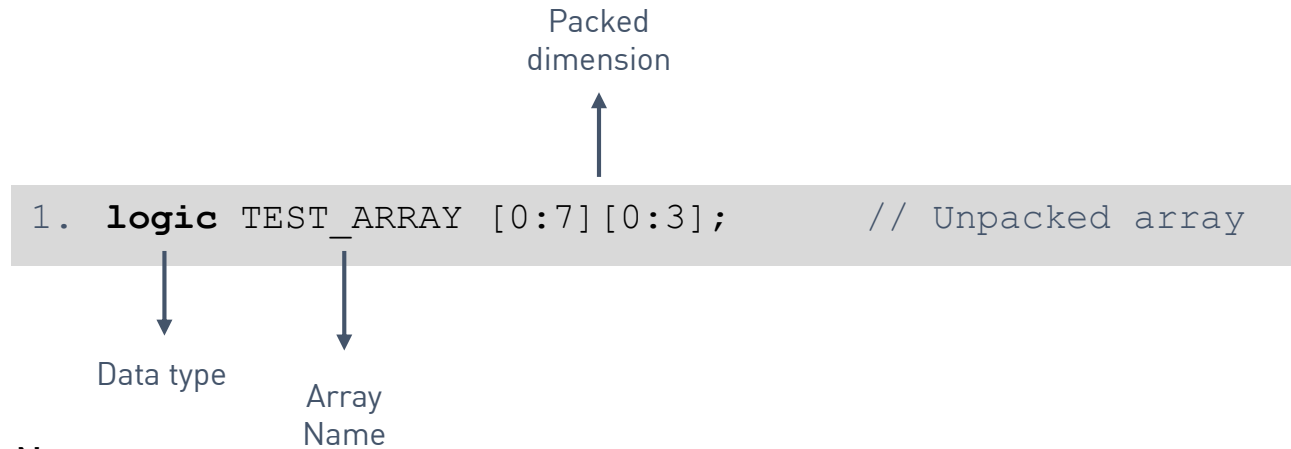
Note

- Unpacked array: Two dimensional array of 1-bit elements ...

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

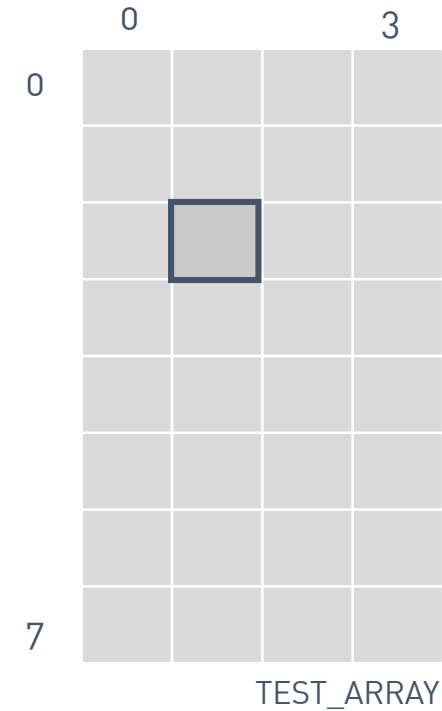
Arrays



Note

- Unpacked array: Two dimensional array of 1-bit elements ...
- Write access:

```
2. assign TEST_ARRAY[2][1] = 1'b1;
```



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Arrays

- It is also possible to have **mixed arrays**. This means that it has at least one packed dimension and one unpacked dimension. Let's start with the following array:

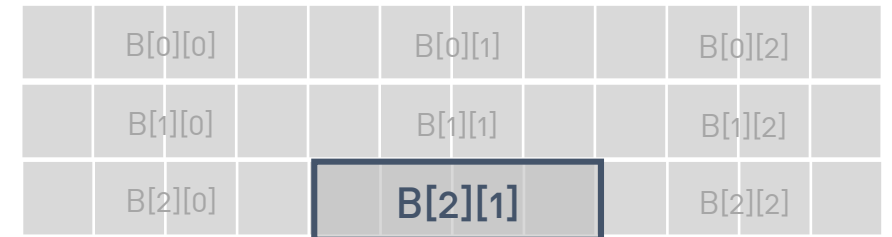
```
1. logic [2:0][3:0] A; // packed array
```

- Let's assume that you would like to have three groups of the previous array.
- Just add an unpacked dimension ...

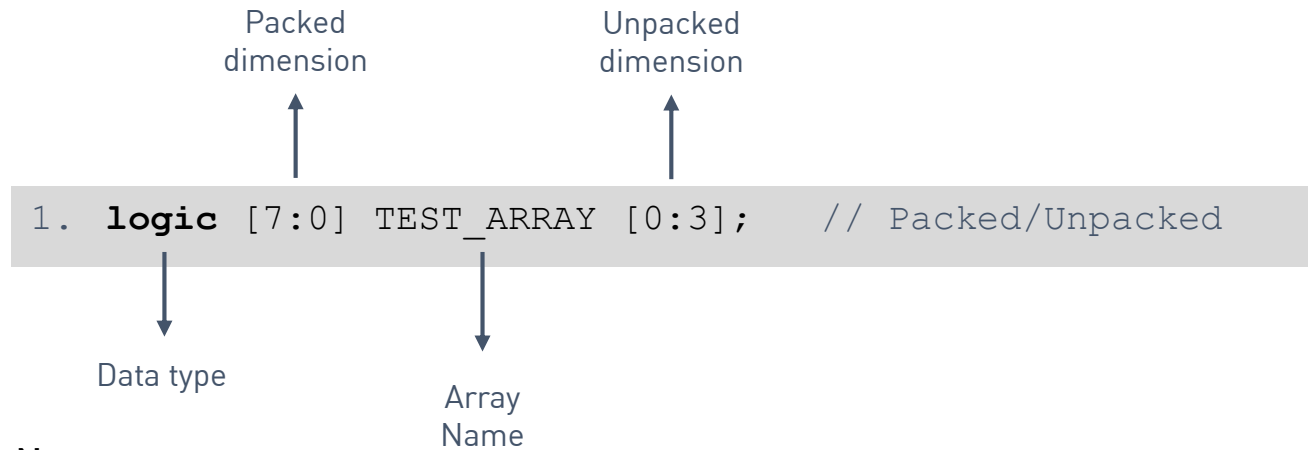
```
2. logic [2:0][3:0] B [0:2];
```

- Accessing a single element:

```
2. assign B[2][1] = 4'b1010;
```

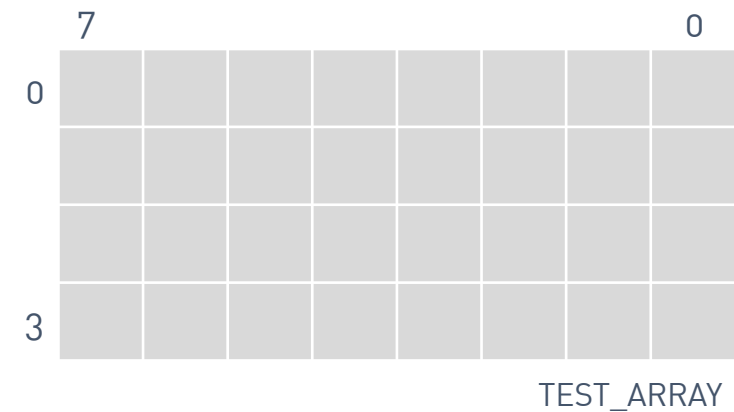


Arrays



Note

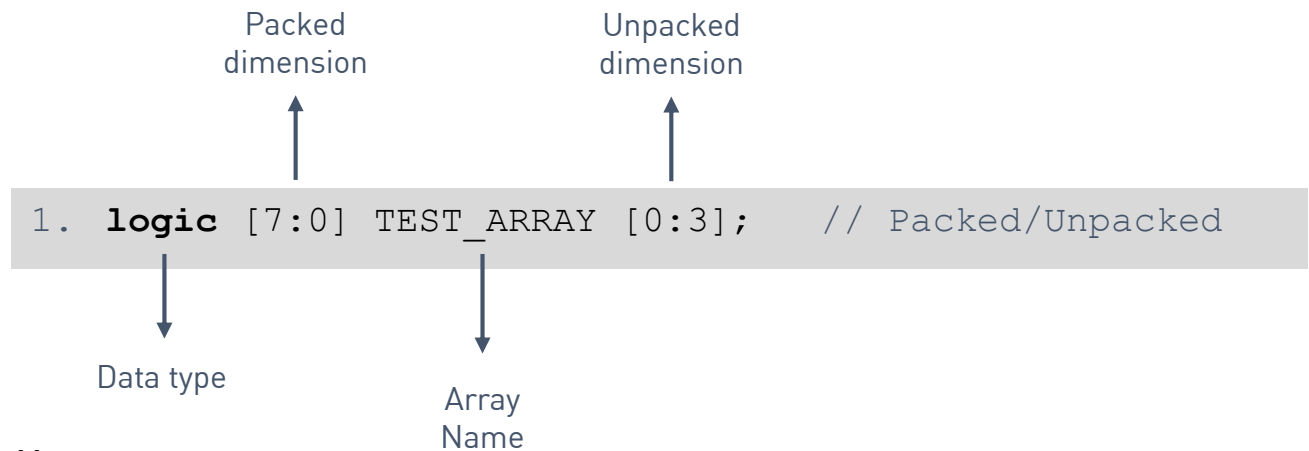
- It is also possible to have mixed arrays. This means that it has at least one packed dimension and one unpacked dimension.



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Arrays



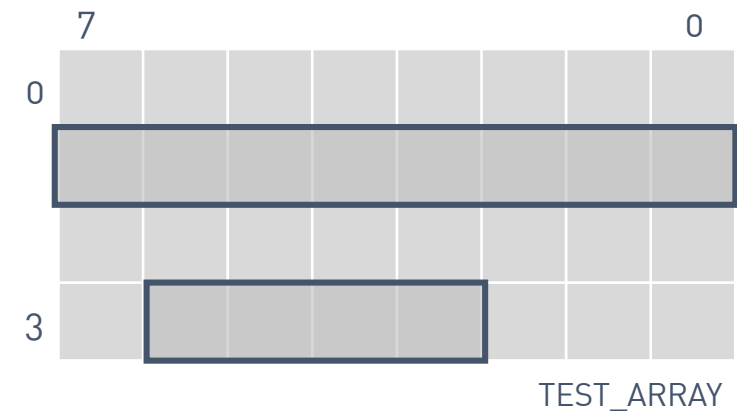
Note

- Write access #1:

```
2. assign TEST_ARRAY[1] = 8'b10101010;
```

- Write access #2:

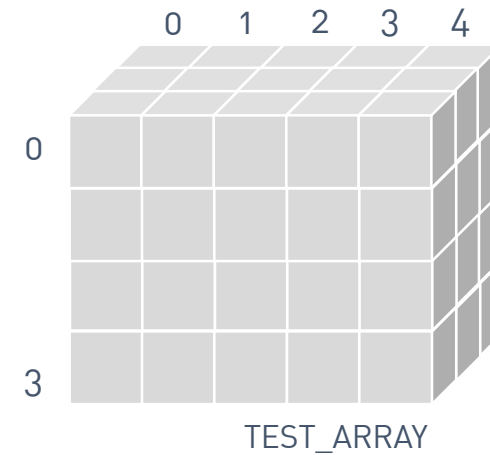
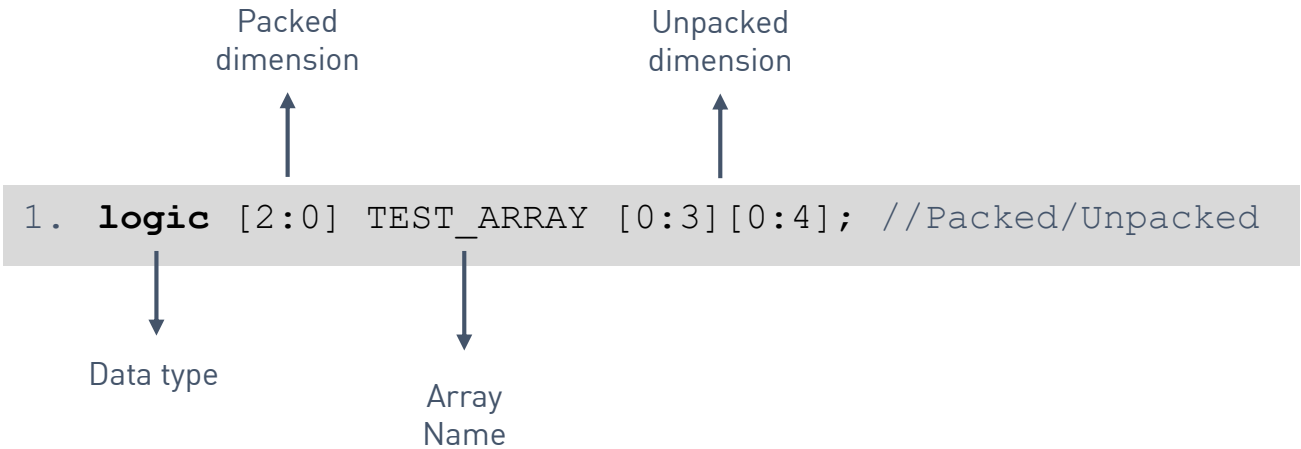
```
2. assign TEST_ARRAY[3][6:3] = 4'b1010;
```



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

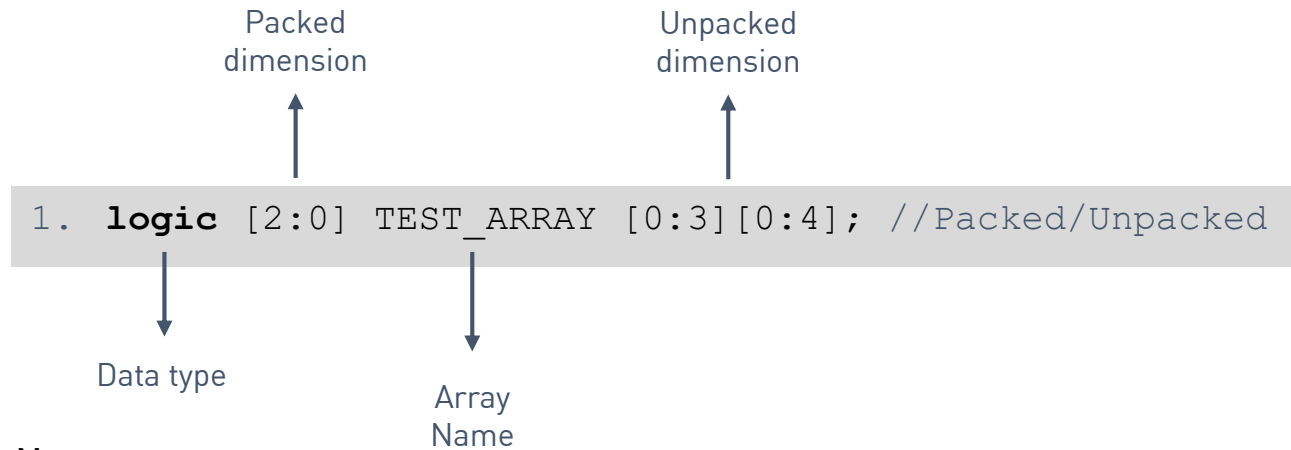
Arrays



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

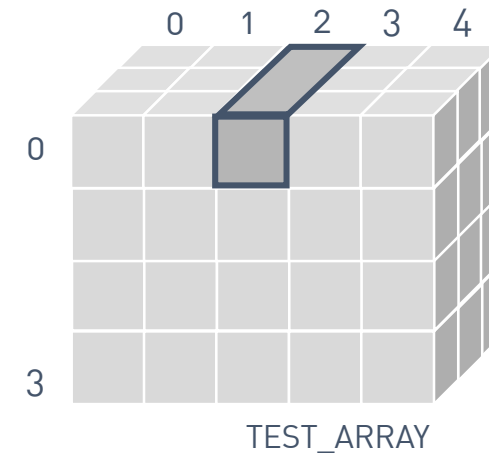
Arrays



Note

- Write access:

```
2. assign TEST_ARRAY[0][2] = 3'b101;
```



Array Reduction Methods

```
1. // data array ...
2. byte data_set[2:0] = `{ 1, 3, 5 };
3. int result;
4.
5. // applying array reduction methods ...
6. assign result = data_set.sum;
7. assign result = data_set.product;
8. assign result = data_set.and;      // bit-wise AND
9. assign result = data_set.or;      // bit-wise OR
10. assign result = data_set.xor;     // bit-wise XOR
```

1/1

[SystemVerilog] Source Code Excerpt

Note

- SystemVerilog Array Reduction methods operate on an **unpacked array** to reduce the array to a single value.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Some more examples

```
1. // 128-element array of 32-bit wide elements
2. logic [31:0] A [127:0];
3. // 2 arrays of 16-bit wide wires indexed from 7 to 0
4. logic [15:0] B[7:0], C[7:0];
5. // 256-entry memory mem_test of 8-bit registers
6. logic [7:0] mem_test [0:255];
7. // two-dimensional array of one bit registers
8. logic data_array [7:0][0:255];
```

1/1

[SystemVerilog] Source Code Excerpt

```
1. mem_test           = 0; // Illegal!
2. data_array[1]       = 0; // Illegal!
3. data_array[1][31:12] = 0; // Illegal!
4. mem_test[1]         = 0; // OK
5. data_array[1][0]    = 0; // OK
```

[SystemVerilog] Source Code Excerpt

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Some more examples

a text file containing hex values

start and end address to specify the section to load (optional)

```
1. $readmemh("hex_memory_file.mem", memory_array, [start_address], [end_address])
2. $readmemb("bin_memory_file.mem", memory_array, [start_address], [end_address])
```

a text file containing hex values

SystemVerilog array

Note

- SystemVerilog allows you to initialize memory from a text file with either hex or binary values.
- Values might be separated by whitespace, tab or a newline

```
1. // Declare a ROM variable
2. logic [15:0] rom [0:3];
3.
4. initial begin
5.     $readmemh("single_port_rom_init.txt", rom);
6. end
```

```
// this is a comment
DEAD
BEEF
DEAD
BEEF
```

[SystemVerilog] Source Code Excerpt

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

SystemVerilog Packages

IEEE Standard 1800-2012

Packages

Looking back ...

- According to the latest Verilog standard (IEEE 1364-2005), declarations of variables, nets, tasks and functions must be declared within a module. Objects declared within a module are local to the module.
- Verilog does not provide a place to make global declarations, such as global functions. A declaration that is used in multiple design blocks must be declared in each block. This requires redundant declarations and is prone to errors

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Packages

```
1. package global_defs; 1/1
2.     enum {FETCH,DECODE,EXECUTE,WRITEBACK} proc_state;
3.     typedef logic [15:0] data_bus;
4.     typedef logic [15:0] ctrl_bus;
5.     ...
6. endpackage
```

[SystemVerilog] Source Code: global_defs.sv

Note

- To enable sharing a user-defined type definition across multiple modules, SystemVerilog adds packages to the Verilog language.
- Packages enable the sharing of parameters, localparms, consts, types, structs, tasks or functions across multiple modules.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Packages

```
1. package global_defs;
2.     enum {FETCH,DECODE,EXECUTE,WRITEBACK} proc_state;
3.     typedef logic [15:0] data_bus;
4.     typedef logic [15:0] ctrl_bus;
5.     ...
6. endpackage
```

1/1

[SystemVerilog] Source Code: global_defs.sv

Note

- To enable sharing a user-defined type definition across multiple modules, SystemVerilog adds packages to the Verilog language. Packages enable the sharing of parameters, localparams, consts, types, structs, tasks or functions across multiple modules.

```
1. module rt_cpu_sys (
2.     input global_defs :: data_bus local_databus,
3.     input global_defs :: ctrl_bus local_ctrlbus,
4.     ...
5. );
```

1/1

1. How to import Packages - #1

Using the double colon scope operator allows the type defined in the global_defs package to be referenced.

[SystemVerilog] Source Code Excerpt: rt_cpu_sys.sv

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Packages

```
1. package global_defs;
2.     enum {FETCH,DECODE,EXECUTE,WRITEBACK} proc_state;
3.     typedef logic [15:0] data_bus;
4.     typedef logic [15:0] ctrl_bus;
5.     ...
6. endpackage
```

1/1

[SystemVerilog] Source Code: global_defs.sv

Note

- To enable sharing a user-defined type definition across multiple modules, SystemVerilog adds packages to the Verilog language. Packages enable the sharing of parameters, localparms, consts, types, structs, tasks or functions across multiple modules.

```
1. import global_defs::*;
2. module rt_cpu_sys (
3.     input data_bus local_databus,
4.     input ctrl_bus local_ctrlbus,
5.     ...
```

1/1

2. How to import Packages - #2

Placing the import instruction outside of the module is known as **global import**. The single types defined in the package can be referenced directly
Package content is accessible for all modules in the design.

[SystemVerilog] Source Code Excerpt: rt_cpu_sys.sv

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Packages

```
1. package global_defs;
2.     enum {FETCH,DECODE,EXECUTE,WRITEBACK} proc_state;
3.     typedef logic [15:0] data_bus;
4.     typedef logic [15:0] ctrl_bus;
5.     ...
6. endpackage
```

1/1

[SystemVerilog] Source Code: global_defs.sv

Note

- To enable sharing a user-defined type definition across multiple modules, SystemVerilog adds packages to the Verilog language. Packages enable the sharing of parameters, localparms, consts, types, structs, tasks or functions across multiple modules.

```
1. module rt_cpu_sys
2. import global_defs::*; (
3.     input data_bus local_databus,
4.     input ctrl_bus local_ctrlbus,
5.     ...
```

1/1

3. **How to import Packages - #3**
Placing the import instruction inside the module is known as **local import**. The single types defined in the package can be referenced directly. The package content is however only accessible from inside the module.

[SystemVerilog] Source Code Excerpt: rt_cpu_sys.sv

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Type casting

```
1. // casting to change size to 10-bits, no truncation 1/1
2. // warning!
3. 10'(x - 2)
4. // multiplication of two real numbers, casting to
5. // type int
6. int'(2.0 * 3.0)
7. // casting to signed
8. signed'(x)
9. // casting to defined enum p_state, converts
10. // number the corresponding value in the enum p_state
11. fruit'(0)
```

[SystemVerilog] Source Code Excerpt

Note

- Verilog is loosely typed, so no significant type checking is done at compile time
- SystemVerilog has more complex data types so it needs to be stricter about type conversions. In conclusion, SystemVerilog requires mechanism for converting between data types: SystemVerilog provides a static cast operator, the apostrophe. Casting allows assignments of variables that may not be otherwise valid.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Operators in SystemVerilog

IEEE Standard 1800-2012

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Operators in SystemVerilog

1. [] bit-select or part select
2. () parenthesis

[SystemVerilog] Operators: selection

1. ! logical negation
2. && logical AND
3. || logical OR

[SystemVerilog] Operators: logical operators

1. ~ bit-wise negation
2. & bit-wise AND
3. | bit-wise OR
4. ^ bit-wise XOR

[SystemVerilog] Operators: bit-wise operators

1. {} concatenation

[SystemVerilog] Operators: concatenation operator

1. {} replication

[SystemVerilog] Operators: replication operators

1. & reduction AND
2. | reduction OR
3. ~& reduction NAND
4. ~| reduction NOR
5. ^ reduction XOR
6. ~^ reduction XNOR

[SystemVerilog] Operators: reduction operators

1. > greater than
2. >= greater than or equal to
3. < less than
4. <= less than or equal to
5. == equal
6. != not equal

[SystemVerilog] Operators: relational operators

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Operators in SystemVerilog contd.

- 1. `==` case equality
- 2. `!=` case inequality

[SystemVerilog] Operators: case equality operators

- 1. `>>` shift right (zero filling)
- 2. `<<` shift left (zero filling)
- 3. `>>>` shift right (sign-extended)
- 4. `<<<` shift left (sign-extended)

[SystemVerilog] Operators: shift operators

- 1. `+` unary (sign) plus
- 2. `-` unary (sign) minus
- 3. `*` multiply
- 4. `/` divide
- 5. `%` modulus
- 6. `+` binary plus
- 7. `-` binary minus

[SystemVerilog] Operators: arithmetic operators

- 1. `?:` conditional

[SystemVerilog] Operators: conditional operators

Introduction to SystemVerilog

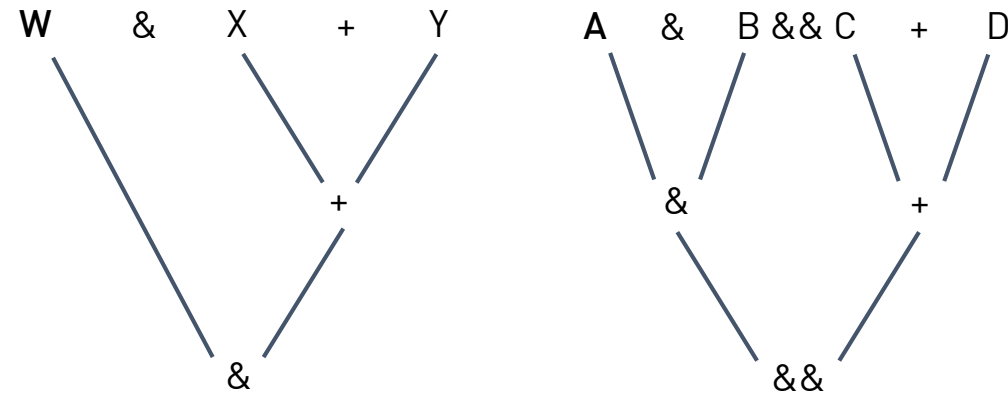
h_da – fbeit - FPGA-based SoC Design

Operator Precedence in SystemVerilog

IEEE Standard 1800-2012

Highest Lowest	~	Not
	*, / , %	Mult, di
	+, -	Add, sub
	<<, >>	Logical shift
	<<<, >>>	Arithmetic shift
	<, <=, >, >=	Comparison
	==, !=	Equal, not equal
	&, ~&	AND, NAND
	^, ~^	XOR, XNOR
	, ~	OR, XOR
? :	Ternary operator	

- Pretty similar to other programming languages ...
- Examples:



- fbeit**
FACULTY OF ELECTRICAL ENGINEERING
AND INFORMATION TECHNOLOGY
Prof. Dr. C. Jakob

Block Names in SystemVerilog

```
1. always_comb begin : write_registers
2.     if(cs && addr[7:6] == 2'b00) begin ctrl_reg
3.         go         <= data_in[15];
4.         ctrl_sel   <= data_in[10:8];
5.         ctrl_val   <= data_in[7:0];
6.         ...
7.     end : ctrl_reg;
8.     else if(cs && addr[7:6] == 2'b01) begin : ram_reg
9.         even      <= data_in[15];
10.        enable <= data_in[14];
11.        wait    <= data_in[10:8];
12.    end : ram_reg
13.    else if(cs && addr[7:6] == 2'b10)
14.        ram_data <= data_in
15. end : write_registers
```

1/1

[SystemVerilog] Source Code Excerpt: rt_cpu_sys.sv

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Tristate Buffers and Bidirectional Ports in SystemVerilog

IEEE Standard 1800-2012

Tristate Buffers

```
1. module tristate (  
2.     input logic d, input logic enable,  
3.     output tri q  
4. );  
5.  
6.     assign q = enable ? d : 1'bz;  
7.  
8. endmodule
```

1/1

[SystemVerilog] Source Code: tristate.sv

Note

- Tristate buffers are switches having three logic states 1, 0 and z (or commonly known as high impedance state).
- Can be mapped to FPGA I/O cells only. Thus tri-state ports can only be used in the top-level design.
- The ability to tri-state is particularly useful when multiple buffer circuits are coupled to the same load since this permits the buffer circuits that are not active in driving the bus to be decoupled therefrom in order to avoid signal contention on the bus.

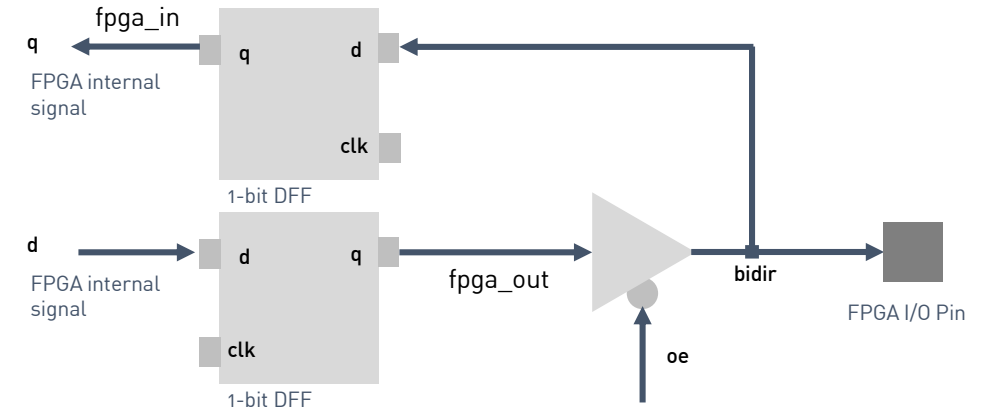
Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Bidirectional ports

```
1. module bir_dir_interface(  
2.     input logic d, oe, clk,  
3.     inout bidir, output logic q  
4. );  
5.  
6. logic fpga_in, fpga_out;  
7.  
8. assign bidir = (oe == 1'b1) ? fpga_out : 1'bz;  
9.  
10. assign q = fpga_out;  
11.  
12. always_ff @(posedge clk) begin  
13.     fpga_in  <= bidir;  
14.     fpga_out <= d;  
15. end  
16.  
17. endmodule
```

1/1



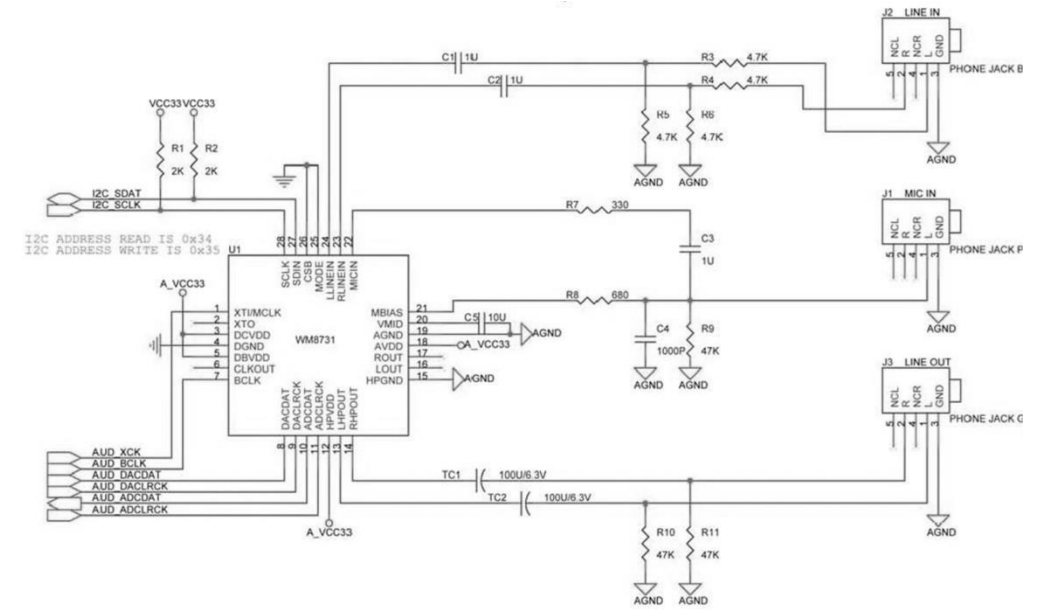
Note

- This example implements a clocked bidirectional pin in SystemVerilog HDL.
- The value of `oe` determines whether `bidir` is an input, feeding in `fpga_in`, or a tri-state, driving out the value `fpga_out`.

[SystemVerilog] Source Code: `bidir_interface.sv`

h_da – fbeit - FPGA-based SoC Design

- Sample rate adjustable from 8 kHz to 96 kHz
- Bit Format
- Loudness
- ...



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Functions

IEEE Standard 1800-2012

Functions

```
1. function <range_or_type> <function_name>;  
2.     <input declaration(s)>  
3.     <variable declaration(s)>  
4.     begin  
5.         // function body ...  
6.         <statements>  
7.     end  
8. endfunction
```

[SystemVerilog] Source Code Excerpt

Note

- Identical code sections are often used in different places in a design. **They are used to improve the readability and to exploit re-usability code**
- Functions are used to group statements to again define a new overall mathematical or logical function.
- **Functions may not contain timing controls such as delays or event control statements.**
- A function returns a single value.
- A function may return a value by assigning the function name, as if it were a variable, or by using the return keyword.

Return value

As stated below, there is an implicit variable is declared inside the function with the same name. The width and type of this implicit variable is declared by <range_or_type>.

Function call

Calls a function, which returns a value for use in an expression

```
1. var = <function_name> (Argument(s));
```

[SystemVerilog] Source Code Excerpt

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Functions contd.

```
1. function logic [7:0] reverse_bits; 1/1
2.   input logic [7:0] byte;
3.   integer i;
4.   begin
5.       for(i = 0; i < 8; i = i + 1)
6.           reverse_bits[7-i] = byte[i];
7.   end
8. endfunction
```

type and range of the implicitly generated return value

input argument

local variable

the begin/end statement is actually just necessary in case of grouping multiple statements

[SystemVerilog] Source Code Excerpt

Note

- As illustrated above, functions may implement (only) combinatorial behaviour.
- They compute a value on the basis of the present value of the input arguments and return a single value.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Functions contd.

Input argument

```
1. function logic [7:0] shift_left(logic [7:0] data); 1/1
2.     begin
4.         shift_left = {data[6:0],1'b0};
5.     end
6. endfunction
```

[SystemVerilog] Source Code Excerpt

Note

- Input arguments can be declared in this form as well.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Functions – Gotcha!

```
1. function signed logic [7:0] signed_op (logic [7:0] data); 1/1
2.     begin
4.         ...
5.     end
6. endfunction
```

[SystemVerilog] Source Code Excerpt

Note

- Be careful when using signed data types!
- The upper declaration states that the return value is a signed type.
- This is different to a variable declaration, where the signed keyword appears after the type:

```
1. logic signed [7:0] my_variable;
```

[SystemVerilog] Source Code Excerpt

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Functions contd.

```
1. module function_example_1(
2.     input logic [7:0] d,
3.     output logic [7:0] q
4. );
5.
6. function logic [7:0] reverse_bits;
7.     input logic [7:0] byte;
8.     integer i;
9.     for(i = 0; i < 8; i = i + 1)
10.         reverse_bits[7-i] = byte[i];
11.
12. endfunction
13.
14. always_comb
15.     q = reverse_bits(d);
16.
17. endmodule
```

1/1

Functions are declared within a parent module ...

and can be called from an always blocks

Alternatively:

Call a function from a continuous assignment or another function

```
1. assign q = reverse_bits(d);
```

[SystemVerilog] Source Code Excerpt

[SystemVerilog] Source Code: function_example_1.sv

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Functions

```
1. function <range_or_type> <function_name>;  
2.     <input declaration(s)>  
3.     <variable declaration(s)>  
4.     begin  
5.         // function body ...  
6.         <statements>  
7.     end  
8. endfunction
```

1/1

[SystemVerilog] Source Code Excerpt

Note

- Functions may be used if there are no delay, timing, or event control constructs
- Functions may be used if only a single value needs to be returned.
- Functions may be used if there is at least one input argument
- Functions may be used if there are no output or inout arguments
- Functions may be used if there are no non-blocking assignments

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Tasks

IEEE Standard 1800-2012

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Tasks

```
1. task <task_name>;  
2.     <i/o declarations>  
3.     <variable and event declarations>  
4.     begin  
5.         // task body ...  
6.         <statements>  
7.     end  
8. endfunction
```

1/1

[SystemVerilog] Source Code Excerpt

Note

- A task is similar to a function, but unlike a function it has both input and output ports. Therefore tasks do not return values.
- Tasks can be used for modelling both combinational and sequential logic.
- A task can contain timing controls. In addition to that it can call other tasks and functions.
- A task can have zero, one, or more arguments. Values are passed to and from a task through arguments. The arguments can be input, output, or inout.

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Tasks

```
1. task reverse_bits;  
2.     input logic [7:0] byte;  
3.     output logic [7:0] q;  
4.     integer i;  
5.     begin  
6.         for(i = 0; i < 8; i = i + 1)  
7.             q[7-i] = byte[i];  
8.     end  
9. endtask
```

1/1

[SystemVerilog] Source Code Excerpt

Note

- As it is the case with functions, tasks are declared within a parent module as well ...

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Tasks

```
1. task SEND_ACKNOWLEDGE;           // semicolon needed
2.     begin                         // begin-end necessary
3.     // Send an acknowledge
4.     tb_ack = 1;
5.     @(posedge tb_local_clock);
6.     # 1;
7.     tb_ack = 0;
8.     @(posedge tb_local_clock);
9.     # 1;
10.    end                           // begin-end necessary
11. endtask                         // no semicolon
```

1/1

Here, the task performs a write access to a global variable `tb_ack`

[SystemVerilog] Source Code Excerpt

Note

- Tasks are often used for testbench scheduling purposes.
- In addition to local variables, tasks can access (reading/writing) global variables as well (defined in the module that is using this task)..